Efficient Attention and Head-Representation Design

pig7selene

September 5, 2026

Abstract

Dense attention has two distinct efficiency problems: executing its quadratic interactions without excessive memory traffic, and retaining the Key/Value state that autoregressive decoding repeatedly reads. This chapter separates these concerns. It develops FlashAttention as an exact, IO-aware execution algorithm based on tiling and online Softmax, then compares Multi-Head Attention, Multi-Query Attention, Grouped-Query Attention, and Multi-Head Latent Attention as architectural choices for representing Key/Value information. The resulting distinction clarifies why kernel optimization and KV-state compression are complementary rather than competing interventions.

1 Introduction

Chapter 3 derived causal Self-Attention as a tensor program: queries compare with eligible keys, the resulting logits are normalized, and the probabilities aggregate values. That derivation identifies the mathematical function but does not determine how a device should execute it or how a deployed model should represent the state left behind by earlier tokens. Those are separate questions with a common practical consequence: attention can be limited by moving data even when its nominal floating-point operation count appears manageable.

This chapter therefore studies two layers of design. **FlashAttention** changes the execution algorithm for the same dense attention function. It reduces traffic between slow and fast GPU memory by processing the computation in tiles and never materializing the full attention matrix in high-bandwidth memory (HBM). **MHA**, **MQA**, **GQA**, and **MLA**, by contrast, are attention-architecture choices. They determine how many Key/Value (KV) representations exist, or how those representations are compressed, and hence change persistent KV Cache state. A model using GQA can still use FlashAttention; neither choice subsumes the other.

Let B be batch size, T sequence length, L the number of Transformer blocks, H_q the number of Query heads, H_{kv} the number of KV heads, and d_h the head dimension. The symbols follow Chapter 3 except that the more explicit H_q and H_{kv} make head sharing easier to compare. We write b for bytes per stored scalar. Chapter 23 distinguishes Prefill from Decode, Chapter 24 derives KV Cache accounting, and Chapter 29 explains why the active performance bottleneck must be measured rather than inferred from one complexity term alone.

2 Attention Cost Is More Than FLOPs

For one head, Scaled Dot-Product Attention first forms scores and then applies those weights to values:

$$S = \frac{QK^T}{\sqrt{d_h}}, \quad P = \mathrm{softmax}(S), \quad O = PV. \tag{1}$$

For a length-T sequence, the two matrix products in Equation 1 require arithmetic proportional to $T^2 d_h$. Across heads, the conventional dense-attention contribution is proportional to $BT^2 H_q d_h$. The all-pairs dependency is mathematical: every legal query-key pair can affect an output. An exact kernel cannot make that dependency linear in T.

Hardware time nevertheless depends on more than arithmetic work. A kernel reads and writes tensors, moves them through a memory hierarchy, launches work, and sometimes synchronizes intermediate results. An implementation can have the same result and nearly the same nominal FLOPs as another while running much faster because it moves fewer bytes over the limiting boundary. This is the IO-aware perspective used by FlashAttention [1].

2.1 The Materialization Problem

The following conceptual execution is useful even though framework kernels may fuse some of its steps:

$$Q, K \to S = QK^T \to P = \mathrm{softmax}(S) \to O = PV. \tag{2}$$

If S and P are stored as separate dense tensors, each has roughly BH_qT^2 elements. The device may write scores to HBM, read them to normalize rows, write probabilities, and read probabilities again for the value product. These intermediate transfers can dominate the useful matrix arithmetic at relevant shapes. The issue is not that S is conceptually invalid; it is that a full $T \times T$ representation is a poor execution boundary when it must repeatedly cross a slow memory boundary.

The GPU hierarchy gives the distinction its force. Registers and on-chip SRAM, often exposed partly as shared memory, are small but fast. HBM or global device memory is much larger but slower to access. A high-level model need not depend on particular bandwidth numbers to use this principle: reuse data in fast storage while it is needed, and avoid writing a large intermediate merely because a step-by-step algebraic presentation introduced it.

3 FlashAttention: Exact IO-Aware Execution

FlashAttention partitions Q, K, and V into blocks that fit in fast on-chip storage. For one query block, it streams successive Key/Value blocks, computes local scores, updates rowwise normalization statistics and output accumulators, and writes only the completed output block. The computation can be summarized as

$$\text{load a Q tile} \to \text{stream K/V tiles} \to \text{update row statistics and output} \to \text{write O tile}. \tag{3}$$

The full probability matrix is not materialized in HBM. The result remains dense attention: every unmasked query-key interaction is still considered. FlashAttention is therefore not an approximate-attention method; up to ordinary floating-point evaluation order, it computes the same mathematical function as Equation 1 while changing the schedule of reads, writes, and reductions [1].

3.1 Online Softmax

Tiling introduces a genuine normalization problem. A Softmax row depends on all its logits, but a query tile sees keys only one block at a time. Consider one row whose already processed logits form set A and whose newly processed block forms set C. Define the partial maxima, normalization factors, and unnormalized value accumulators as

$$m_A = \max_{j \in A} s_j, \quad \ell_A = \sum_{j \in A} \exp\big(s_j - m_A\big), \quad n_A = \sum_{j \in A} \exp\big(s_j - m_A\big) v_j, \tag{4}$$

with analogous quantities m_C, ℓ_C, and n_C for the new block. Let $m = \max(m_A, m_C)$. The combined state is

$$\ell = \exp(m_A - m)\ell_A + \exp(m_C - m)\ell_C, \quad n = \exp(m_A - m)n_A + \exp(m_C - m)n_C, \quad o = \frac{n}{\ell}. \tag{5}$$

The merge in Equation 5 is exact in real arithmetic. When a later block has a larger maximum, the earlier partial sum and value accumulator are rescaled to the new stable reference. The final ratio equals the ordinary Softmax-weighted value sum, but neither the global score row nor the global probability row needs to be stored. Max subtraction also controls exponential range, connecting this execution method to Chapter 8′s discussion of numerically stable Softmax.

3.2 Memory and Later Improvements

Dense attention's **mathematical interactions** remain quadratic. FlashAttention does not remove the T^2 score relationships or promise that every workload becomes compute-bound. Its key benefit is reducing the additional intermediate-memory footprint and HBM traffic associated with materializing scores and probabilities. Backward computation can recompute local quantities rather than retaining the full attention matrix, trading controlled arithmetic for less activation storage.

FlashAttention-2 retains this IO-aware exactness while improving work partitioning, parallelism, and non-matrix-multiplication overhead [2]. The reusable lesson is not a version-specific kernel recipe. Execution performance depends on how work is divided across available processors and on whether the implementation makes productive use of the memory hierarchy. Shapes, masking form, dtype, dropout, and hardware determine whether a particular kernel path realizes a large gain; a familiar kernel name is not itself a performance result.

4 Head-Representation Designs

Chapter 3 introduced attention with a general mapping from Query heads to KV heads. Here the central distinction is restated compactly. MHA assigns independent Key and Value heads to every Query head. MQA keeps many Query heads but shares one Key head and one Value head. GQA shares KV heads within groups of Query heads. These are not three execution algorithms for the same checkpoint: they are architectural configurations of the learned K and V projections.

4.1 MHA, MQA, and GQA

For ordinary Multi-Head Attention (MHA),

$$H_{\mathrm{kv}} = H_q. \tag{6}$$

Each Query head has a distinct Key head and Value head. This is the standard multi-head form developed in Chapter 3. Multi-Query Attention (MQA) preserves H_q Query heads but uses one shared Key and one shared Value head:

$$H_{\mathrm{kv}} = 1. \tag{7}$$

The sharing reduces cached state and repeated Decode-time KV reads, which motivated MQA as a faster Transformer decoding design [3]. Its cost is an aggressive restriction: all Query heads must address the same per-token Key/Value representation.

Grouped-Query Attention (GQA) supplies an intermediate choice:

$$1 < H_{\mathrm{kv}} < H_q. \tag{8}$$

Query heads are partitioned into groups, and every group shares one Key head and one Value head. GQA thereby retains more independent KV representations than MQA while reducing them relative to MHA. Ainslie et al. introduce this interpolation and show why it is a useful quality-efficiency design space rather than a binary MHA-versus-MQA decision [4].

4.2 KV Cache Consequences

For dense decoder-only attention, the ideal persistent cache payload has the familiar form

$$M_{\mathrm{KV}} \approx 2BLTH_{\mathrm{kv}}d_h b. \tag{9}$$

The factor two accounts for Keys and Values. Decreasing H_{kv} directly decreases cache capacity and cache-bandwidth demand even though H_q can remain fixed. It does not reduce the number of Query rows or eliminate dense query-key interactions. This is why head sharing is particularly consequential during Decode, where each newly generated Query reads a growing history of cached Keys and Values. Chapter 24 gives the broader lifecycle, paging, sharing, and quantization consequences of this equation.

Design	H_{kv}	KV representation	Primary Decode consequence
MHA	H_q	One independent KV head per Query head.	Largest conventional KV Cache and KV-read volume.
MQA	1	One shared KV head for all Query heads.	Minimum head-count cache footprint; strongest sharing constraint.
GQA	$1 < H_{\text{kv}} < H_q$	One KV head per Query-head group.	Intermediate cache and bandwidth trade-off.

Table 1: MHA, MQA, and GQA differ in the learned organization of Key/Value representations. The Query-head count need not change when the KV-head count is reduced.

5 Multi-Head Latent Attention

Multi-Head Latent Attention (MLA) targets the same broad Decode bottleneck with a different compression mechanism. Instead of only reducing the number of full KV heads, it stores a lower-dimensional latent representation from which content-related Key and Value information is reconstructed or used by head-specific projections. MLA was introduced in the DeepSeek-V2 technical report as a KV-compression architecture [5].

For an input hidden state h_t, a schematic latent mapping is

$$c_t = W_c h_t, \quad c_t \in R^{d_c}, \quad d_c \ll 2H_{\text{kv}} d_h. \tag{10}$$

The inequality expresses the design objective, not a universal MLA definition: the latent width d_c is chosen to be much smaller than storing independent full Key and Value vectors. Subsequent projections can derive attention-specific content representations from c_t. A conceptual form is

$$k_t^{\text{content}} = W_K^{\text{up}} c_t, \quad v_t = W_V^{\text{up}} c_t. \tag{11}$$

The constructions in Equation 10 and Equation 11 convey the compression principle without claiming that every MLA implementation has identical factorization, ranks, or kernels. The serving cache can retain the latent state rather than all expanded KV vectors when the attention implementation is designed to use it. That changes both cache representation and the computation that reads or reconstructs it.

5.1 Positional Components and RoPE

Content compression interacts delicately with position encoding. Chapter 3 explains that RoPE rotates query and key coordinates so their inner product carries relative displacement. A low-rank latent representation that is convenient for content projections need not preserve the same positional behavior after arbitrary reconstruction. MLA-style architectures can therefore keep a positional Key component separate from compressed content state and combine it with a matching positional Query component during score formation.

Conceptually, one may write

$$q_t = [q_t^{\text{content}}; q_t^{\text{pos}}], \quad k_j = \left[k_j^{\text{content}}; k_j^{\text{pos}}\right], \tag{12}$$

where only the positional components receive the appropriate RoPE transformation. This is an architectural compatibility mechanism, not an instruction to split every Transformer head this way. Its importance is general: a cache-compression design must preserve the positional convention used by the checkpoint, rather than treating position information as an interchangeable post-processing detail.

5.2 MLA Compared with Head Sharing

MQA and GQA reduce **how many** independently stored KV heads exist. MLA compresses **what each token stores** into a lower-dimensional latent state. Both can reduce KV Cache capacity and Decode bandwidth, but their quality, projection cost, kernel requirements, and compatibility constraints differ. A comparison should consequently state the representation, cache dtype, sequence length, batch regime, and serving kernels rather than ranking the names in isolation.

Design	Stored per-token state	Main compression mechanism	Important engineering condition
MHA	Full KV vectors for every head.	None beyond ordinary head factorization.	Conventional kernels and largest cache payload.
MQA	One shared full KV pair.	Share KV across all Query heads.	Quality must tolerate maximal sharing.
GQA	One full KV pair per group.	Share KV within Query-head groups.	Group count must match checkpoint configuration.
MLA	Compressed latent KV state, plus required positional state.	Low-dimensional latent representation and reconstruction.	Architecture-specific projections, RoPE handling, and kernel support.

Table 2: Architectural KV-compression strategies. MLA is not merely GQA with fewer heads: it changes the representation retained for each token.

6 Kernel Optimization and Architecture Are Complementary

FlashAttention changes neither H_{kv} nor the learned meaning of a Key or Value. It reduces intermediate attention traffic during Prefill and training by executing the same dense operation in a different order. MQA, GQA, and MLA alter the checkpoint's attention representation and reduce persistent KV state or its movement during Decode. The distinctions are summarized separately because a single table that calls one design "more efficient" would confuse different resources.

Execution choice	Mathematical attention function	Primary resource changed	Primary phase
Conventional dense execution	Dense attention	May materialize large score/probability intermediates.	Training and Prefill.
FlashAttention-style execution	Same dense attention	HBM traffic and intermediate-memory footprint.	Training and Prefill; supported Decode shapes.

Table 3: FlashAttention is an execution choice, not a head-sharing architecture. A model with MHA, MQA, GQA, or MLA can require a separate decision about its attention kernel.

During training, FlashAttention can reduce activation-memory pressure and improve attention-kernel utilization, while MQA, GQA, or MLA are fixed architectural choices learned with the model. During inference, FlashAttention may improve Prefill and compatible attention computations; MQA, GQA, and MLA primarily change persistent cache capacity and Decode-time state movement. End-to-end gains can still be limited by projections, model weights, scheduling, communication, or other system costs. Chapter 29′s bottleneck-first method is therefore the correct way to evaluate a combined design.

7 Failure Modes and Practical Limits

The clean conceptual boundaries above make common errors easier to diagnose. FlashAttention can be unavailable for a dtype, mask layout, head dimension, hardware generation, or runtime path. Very short sequences can leave tile or launch overhead more visible than saved HBM traffic. An incorrect online Softmax merge can cause numerical error or a silent mismatch with a dense reference, especially

when masking and reduced precision are involved. It is not enough to report that a kernel was selected; a test must compare its permitted outputs and gradients with an appropriate reference tolerance.

Head-sharing designs have a different risk profile. Treating H_q as H_{kv} misstates cache capacity for MQA and GQA. A checkpoint's head mapping, projection shapes, and positional convention cannot be changed at serving time by merely changing a configuration field. Aggressive KV sharing can reduce model quality, and a nominal cache reduction may not improve latency if weights, scheduler delay, or another operation dominates. MLA adds projection, representation, and positional-encoding complexity; an implementation that expands and stores its full reconstructed KV state would forfeit much of the intended cache benefit.

8 Implementation Contracts

An attention-kernel contract must state the score scaling, causal or padding-mask semantics, supported tensor layouts, dtype and accumulation precision, dropout behavior in training, and the numerical tolerance against a reference dense implementation. Tests should include variable sequence lengths, masked rows, boundary tiles, and long rows whose maximum occurs in a later Key block. They should verify both forward outputs and, when training is supported, gradients. Memory reports should separate persistent KV Cache from temporary attention workspace; otherwise FlashAttention's intermediate-memory benefit and a head-design cache benefit will be conflated.

An architectural cache contract must record H_q, H_{kv}, d_h, layer count, cache dtype, the exact model revision, and positional convention. Its expected ideal payload should agree with Equation 9 before allocator rounding and metadata. For MQA and GQA, tests should verify the Query-to-KV head mapping and cache tensor shape. For MLA, they must additionally verify the latent projection, any reconstruction or absorbed-projection algebra, decoupled positional components, and equivalence to the model's specified attention rule. Benchmarking must state Prefill and Decode lengths, batch concurrency, kernel path, and quality checks; a byte-count reduction alone is not a serving result.

9 Summary

Dense attention combines quadratic query-key interactions with an execution problem: a naive schedule can move large score and probability intermediates through HBM. FlashAttention preserves the attention function while using tiles and online Softmax to avoid materializing the full attention matrix in slow memory. It reduces intermediate-memory demand and IO, but it does not make dense attention's all-pairs arithmetic linear.

MHA, MQA, GQA, and MLA address a separate bottleneck: the Key/Value state stored and read across autoregressive steps. MQA and GQA share full KV heads at different granularities, whereas MLA compresses KV information into a latent representation and requires compatible positional handling. FlashAttention can be paired with each of these designs. Chapter 31 then considers the separate architectural choice of computing only a structured subset of attention edges. The useful systems question is consequently not which label is globally best, but whether the measured workload is constrained by attention IO, persistent KV state, computation, or another component of the serving path.

References

[1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems 35*, Curran Associates, Inc., 2022, pp. 16344–16359. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract.html

[2] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2307.08691

[3] N. Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need," *arXiv preprint arXiv:1911.02150*, 2019, [Online]. Available: https://arxiv.org/abs/1911.02150

[4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 4895–4901. doi: 10.18653/v1/2023.emnlp-main.298.

[5] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," *arXiv preprint arXiv:2405.04434*, 2024, doi: 10.48550/arXiv.2405.04434.